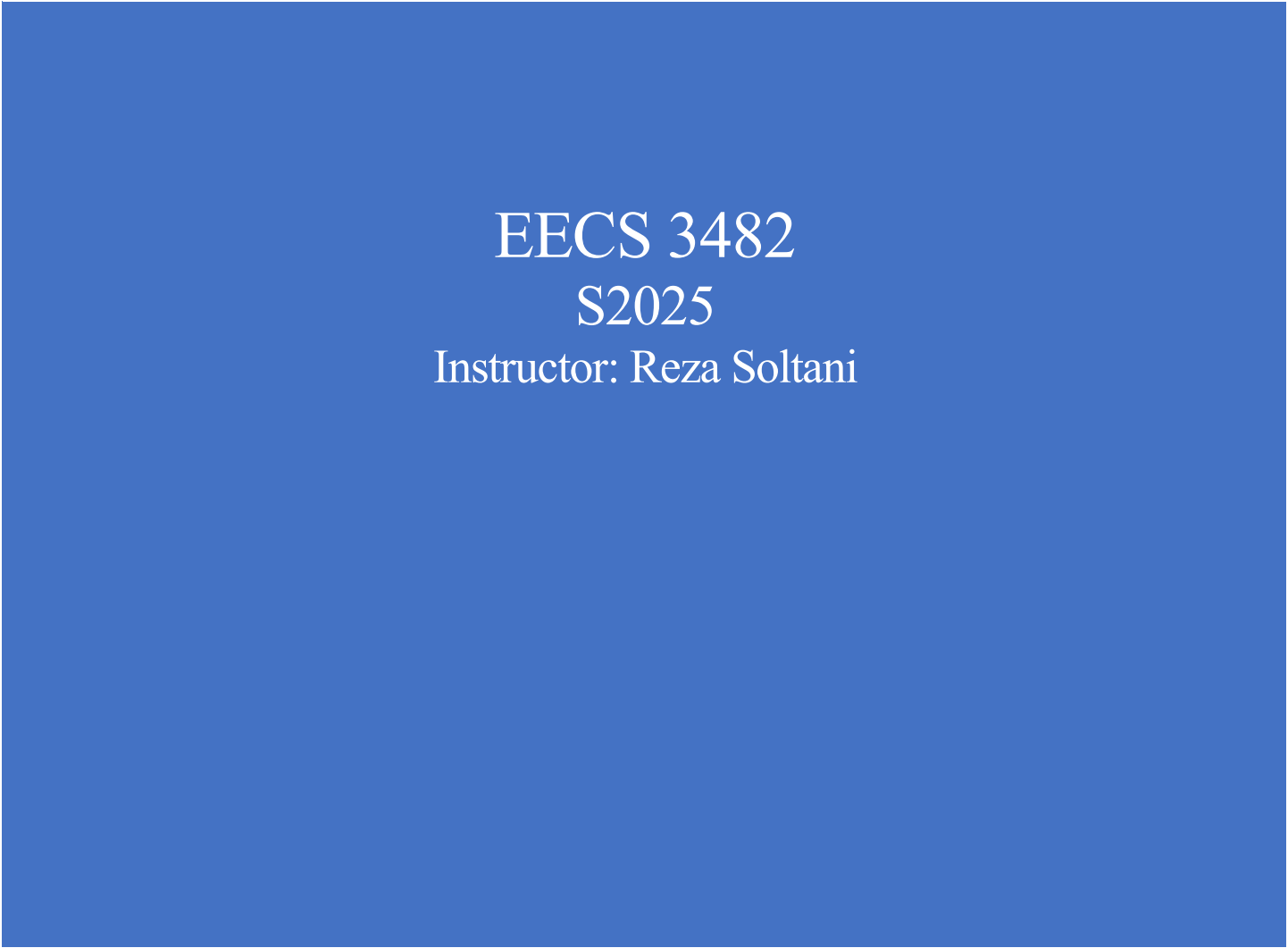




Lab 1 - Symmetric and Asymmetric Cryptography



EECS 3482
S2025
Instructor: Reza Soltani

Lab Objectives

By the end of this lab, students will:

1. Understand the principles of symmetric and asymmetric cryptography.
2. Use **OpenSSL** to perform encryption, decryption, and key management.
3. Explore digital signatures for message integrity.
4. Analyze key sizes, encryption speed, and trade-offs between cryptographic methods.
5. Develop a lightweight encryption client in Python

Lab Requirements

- **Tools/Environments:**
 - Lab Environment
 - OpenSSL
 - Python Interpreter
- **Prerequisites:**
 - Basic knowledge of Linux command-line interface (CLI).
 - Familiarity with cryptographic terminology.
 - Basic understanding of Python language

Lab Format

For every step of your labs, including this lab and all future labs in the course, you must provide a clear and **legible screenshot** whenever you use the command prompt.

Before taking the screenshot, and either before or after completing the step, you **must** enter the following command so that your **username**, **full name**, and **timestamp** are visible in the screenshot:

```
echo " Step completed by $(whoami), name: <first> <last>, on $(date)"
```

Note, replace *<first>* and *<last>* with your own first name and last name before running the command.

For example, a student with the name *John Smith* and username *JSUser1* should run the following command:

```
echo "Step completed by $(whoami), name: john smith, on $(date)"
```

to get the following result:

```
Step completed by JSUser1, name: John Smith, on Sat 17 May 2025 12:48:05 EDT
```

Note, there are different kinds of shell interface within Unix/Linux systems. Depending on the environment you are in (such as the *eeecsvm*'s kali environment which uses **ksh** instead of **bash** shell), you may have to enter the following modified command:

```
echo " Step completed by `whoami`, name: <first> <last>, on `date`"
```

or alternatively, enter **bash** in the command prompt to switch to *bash* shell before entering the original *echo* command.

Also note: If you are working on a Windows machine, you must use an equivalent command such as:

```
echo Step completed by %USERNAME%, name: <first> <last>, on %DATE% %TIME%
```

If a lab step does not include the above command or a clear screenshot, marks will be deducted.

Lab Tasks

Part 1: Symmetric Encryption (AES)

Objective: Learn how to encrypt and decrypt data using a symmetric algorithm.

1. Key Generation and Encryption

- On a **Linux** terminal, generate a random **256-bit symmetric key**:

```
openssl rand -base64 32 > symmetric_key.key
```

- Verify the key:

```
cat symmetric_key.key
```

- Encrypt a sample plaintext file which only includes **your first and last name** (separated by a space) and call it “plaintext.txt”, using **AES-256-CBC**:

```
openssl enc -aes-256-cbc -salt -in plaintext.txt -out  
encrypted_file.enc -pass file:symmetric_key.key
```

- **Question 1:** What is the role of the `-salt` flag in AES encryption?

2. Decryption

- **Write** the command to decrypt the encrypted file (`encrypted_file.enc`) and verify its content. Call your file: `decrypted_file.txt`
- Compare the original and decrypted files:

```
diff plaintext.txt decrypted_file.txt
```

- **Question 2:** What happens if you attempt to decrypt the file with the wrong key?

3. Performance and Key Size Experiment

- Use different AES key sizes (128-bit, 192-bit, 256-bit) to encrypt the same file. Measure encryption time using the `time` command:

```
time openssl enc -aes-128-cbc -in plaintext.txt -out enc128.enc -  
pass file:symmetric_key.key  
time openssl enc -aes-192-cbc -in plaintext.txt -out enc192.enc -  
pass file:symmetric_key.key  
time openssl enc -aes-256-cbc -in plaintext.txt -out enc256.enc -  
pass file:symmetric_key.key
```

- **Question 3:** How does the encryption time vary with key size? What trade-offs do you observe? Is the 'time' command precise enough to show you the difference between the three commands?

Part 2: Asymmetric Encryption (RSA)

Objective: Use RSA to encrypt/decrypt a file and generate a digital signature.

1. Generate RSA Key Pairs

- Generate a **2048-bit RSA key pair**:

```
openssl genrsa -out private_key.pem 2048
openssl rsa -pubout -in private_key.pem -out public_key.pem
```

- Verify the keys:

```
cat private_key.pem
cat public_key.pem
```

2. Encrypt and Decrypt a File

- Use the same `plaintext.txt` from Part 1 in the following steps.
- **Write** the command to encrypt the plaintext file using the **public key**. Call your file `rsa_encrypted.enc`
- **Write** the command to decrypt the file using the **private key**. Call your output file `rsa_decrypted.txt`
- Verify the decrypted file:

```
diff plaintext.txt rsa_decrypted.txt
```

- **Question 4:** Why is RSA encryption slower compared to AES?

3. Digital Signatures

- Generate a hash of the file and sign it with the private key:

```
openssl dgst -sha256 -sign private_key.pem -out signature.bin
plaintext.txt
```

- **Write** the command to verify the signature using the public key.
- **Question 5:** What role does hashing play in digital signatures?

Part 3: Python Application

1. Write a small Python application that performs **AES-256** encryption and decryption on a given string variable. The string consists of your first name followed by your last name. For example if your name is John Smith, the string would be exactly “John Smith”.
2. Name your program encrypt.py
3. Ensure the Python script runs using the command Python encrypt.py or Python3 encrypt.py
4. Your program should not take any command line arguments. Make sure you are using Python 3.0+.
5. When executing, the program should print out three lines:
 - plaintext string
 - the encrypted valued in BASE64 format
 - the plaintext string after the encrypted value is decrypted.

These 3 values should be printed on separate lines. Therefore, after executing your program, there should be 3 lines of output with the first and the third lines being exactly the same. Copy and paste the result of your program in a file called output.txt.

6. Ensure your program uses known Python libraries with the correct parameters and a good (i.e. random) key and a proper IV parameter for encryption/decryption.
7. Ensure you use the CBC mode of operations for AES encryption.
8. Verify proper padding of type PKCS#7 is added during your encryption.
9. Ensure your program has adequate comments, especially for the lines involved with the encryption and decryption steps.
10. Test your program before submitting!
11. Submit your source code (**encrypt.py**) along with the output of the program (**output.txt**)

Lab Submission

1. A PDF that includes the commands required for every step, and answers to the questions in Part 1, and 2. **Ensure you follow the instructions under ‘Lab Format’ section to produce proper screenshots for every step.**
2. All the files associated to keys, plaintext, encrypted and decrypted steps for AES and RSA in Part 1 and 2:
 1. `symmetric_key.key`
 2. `plaintext.txt`
 3. `encrypted_file.enc`
 4. `decrypted_file.txt`
 5. `private_key.pem`
 6. `public_key.pem`
 7. `rsa_decrypted.txt`
 8. `rsa_encrypted.enc`
 9. `signature.bin`
3. You do not need to submit the files for part 1-step 3.
4. The Python program and the output of the program from part 3.
 1. `encrypt.py`
 2. `output.txt`
5. Submit all your files through eClass before the deadline! **Late submissions will not be accepted.**